SISTEMAS DE COMUNICACIÓN Y SEGURIDAD INFORMÁTICA • 8VO SEMESTRE

Bastionado Integral (Hardening) y Defensa Activa en Capa 7 de Servidores Nginx con Docker

Mitigación Determinista de OWASP A05:2021 y Ataques DoS HTTP Flood

CAPA 7 OSI | LEAKY BUCKET SHM | DOCKER ISOLATION

Asignatura:	Sistemas de Comunicación y Seguridad Informática
Unidades Integradas:	Semana 5 (Hardening & OWASP) + Semana 10 (Defensa Red L7)
Nivel de Estudio:	8vo Semestre — Ingeniería
Equipo de Trabajo:	3 Integrantes (Expositores 1 a 3)
Entorno Técnico:	Nginx Alpine, Docker Bridge, Apache Benchmark, PowerShell
Estado de Validación:	**Aprobado y Verificado Experimentalmente**

Documento y Manual Oficial de Sustentación para Evaluación Académica

1. Resumen Ejecutivo (Abstract)

El presente informe documenta el diseño, despliegue y análisis forense de una solución de **Bastionado (Hardening)** y **Defensa Activa en Capa 7 (Aplicación)** para servidores web Nginx contenedorizados en Docker. La arquitectura mitiga deterministamente dos vectores críticos de ataque contemporáneos:

1. **Exposición de Metadatos y Vulnerabilidades del Cliente (OWASP A05:2021 - Security Misconfiguration):** Supresión de versión del servidor para neutralizar el *Banner Grabbing*, restricción de verbos HTTP (`GET` / `HEAD`) e inyección de encabezados HTTP mandatorios (`X-Frame-Options` , `X-Content-Type-Options` , `Content-Security-Policy` , `X-XSS-Protection: 0` , `Referrer-Policy` , `Permissions-Policy`) que mitigan ataques de *Clickjacking*, *XSS* y *MIME-Sniffing*.
2. **Denegación de Servicio en Capa de Aplicación (Capa 7 HTTP Flood y Slowloris):** Configuración de zonas de memoria compartida (Shared Memory Zones de 10 MB) con rastreo de IP binaria (`$binary_remote_addr`): algoritmo **Leaky Bucket** con tasa nominal de **5 r/s** y ráfaga `burst=10 nodelay` , límite de **20 conexiones TCP concurrentes por IP** y timeouts estrictos anti-Slowloris (`10s`).

Bajo una prueba de estrés masiva con **Apache Benchmark (1000 solicitudes, concurrencia 50)**, el sistema demostró un comportamiento determinista: en **4.100 segundos** admitió exactamente **31 peticiones legítimas (200 OK)** y rechazó **969 solicitudes excedentes (503 Service Unavailable, 96.9%)**, además de bloquear métodos no permitidos (`POST` → `403`) manteniendo el contenedor estable bajo límites de **128 MB RAM** y **0.50 CPU**.

2. Fase 1: Contextualización y Escenario de Amenaza

Los servidores web perimetrales constituyen la frontera de exposición directa hacia redes inseguras. Los firewalls convencionales de paquetes (Capas 3 y 4 del modelo OSI) resultan inoperantes frente a ataques en Capa 7, ya que el tráfico se encapsula sobre conexiones TCP formalmente completadas (Three-Way Handshake legítimo).

> **Vectores de Ataque Primarios:**
> - **Banner Grabbing y Fingerprinting:** Escaneo automatizado mediante `curl -I` , Nmap o Shodan para extraer cadenas del tipo `Server: nginx/1.25.3` y correlacionar vulnerabilidades conocidas en la base NVD/CVE.
> - **Ataques en el Navegador del Cliente (OWASP A05):** Explotación de la falta de cabeceras restrictivas para incrustar el sitio en marcos invisibles (Clickjacking) o engañar al intérprete MIME del navegador (MIME-Sniffing).
> - **Inundación HTTP (HTTP Flood DoS):** Saturación deliberada de los descriptores de archivo (file descriptors) y procesos worker del servidor mediante solicitudes concurrentes masivas.

3. Fase 2: Fundamentación Teórica

3.1 OWASP Top 10 — A05:2021 (Security Misconfiguration)

- **Supresión de Banner:** La directiva `server_tokens off;` elimina la subversión de Nginx en los encabezados HTTP y en las páginas de error estándar generadas por el motor web.
- **Clickjacking (UI Redressing):** `X-Frame-Options: SAMEORIGIN` y `frame-ancestors 'self'` (en CSP) ordenan al navegador no permitir que el sitio sea incrustado en etiquetas `<iframe>` de otros dominios.
- **MIME-Sniffing (Type Confusion):** `X-Content-Type-Options: nosniff` obliga al cliente a ceñirse al `Content-Type` declarado por el servidor, impidiendo que archivos no ejecutables sean interpretados como scripts JavaScript.
- **Content Security Policy y cabeceras modernas:** `Content-Security-Policy` restringe los orígenes de carga, `X-XSS-Protection: 0` sigue la guía vigente de OWASP junto con CSP, y `Permissions-Policy` bloquea APIs sensibles del navegador.
- **Directiva `always`:** Garantiza que Nginx inyecte las cabeceras de seguridad incluso cuando las respuestas devuelven códigos de error (4xx y 5xx).
- **Restricción de métodos y Anti-Slowloris:** `limit_except GET` bloquea verbos innecesarios (`POST` , `PUT` , `DELETE`), mientras `limit_conn zona_conn 20;` y timeouts de `10s` evitan el agotamiento de sockets por conexiones lentas.

3.2 Algoritmo de Defensa Activa: Leaky Bucket en Memoria Compartida

El módulo `ngx_http_limit_req_module` modela el consumo de recursos como un cubo con una fuga constante.

- **Optimización con `$binary_remote_addr`:** En lugar de almacenar cadenas de texto ASCII de 15 bytes, Nginx almacena la dirección IPv4 en representación binaria de **4 bytes**. En una zona de **10 Megabytes (SHM)**, esto permite monitorear de forma concurrente el estado de más de **160,000 direcciones IP** simultáneas en memoria RAM.
- **Tasa nominal (`rate=5r/s`):** Fija el ritmo de atención a 1 petición cada 200 ms.
- **Búfer de Tolerancia (`burst=10`):** Permite absorber ráfagas legítimas de tráfico transitorio.
- **Política Fail-Fast (`nodelay`):** Atiende los slots disponibles del burst inmediatamente y *descarta al instante* toda petición excedente con código `503 Service Unavailable` sin encolamientos lentos que bloqueen sockets en el kernel.

4. Fase 3 y 4: Arquitectura e Implementación del Stack

4.1 Arquitectura de Red Aislada (Docker Bridge)

Para aislar las mediciones y descartar latencias de redes externas, se desplegó la subred `red-defensa (172.28.0.0/16)`:

- **Servidor Defensor (`172.28.0.10`):** Nginx Alpine con montaje de volúmenes en solo lectura (`:ro`).
- **Cliente Atacante (`172.28.0.20`):** Contenedor Alpine instrumentalizado con `apache2-utils` (Apache Benchmark) y `curl` .

4.2 Archivo de Configuración Maestro: `nginx.conf`

```
events {
    worker_connections 1024;
}

http {
    include       mime.types;
    default_type  application/octet-stream;

    # --- MITIGACIÓN OWASP A05:2021 (HARDENING) ---
    server_tokens off;
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header Content-Security-Policy "default-src 'self'; style-src 'self' 'unsafe-inline'; object-
    add_header X-XSS-Protection "0" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;

    # --- PROTECCIÓN ANTI-SLOWLORIS Y TIMEOUTS ---
    client_header_timeout 10s;
    client_body_timeout   10s;
    send_timeout          10s;
    keepalive_timeout     15s;
    reset_timedout_connection on;
    client_max_body_size  1m;
    limit_conn_zone $binary_remote_addr zone=zona_conn:10m;
    limit_conn_status 503;

    # --- DEFENSA ACTIVA EN CAPA 7 (RATE LIMITING) ---
    limit_req_zone $binary_remote_addr zone=zona_limite:10m rate=5r/s;
    limit_req_status 503;

    server {
        listen 80;
        server_name localhost;

        location / {
            limit_except GET {
                deny all;
            }
            limit_conn zona_conn 20;
            limit_req zone=zona_limite burst=10 nodelay;
            root   /usr/share/nginx/html;
            index  index.html index.htm;
        }

        error_page 503 /custom_503.html;
        location = /custom_503.html {
            root /usr/share/nginx/html;
            internal;
        }
    }
}
```

5. Fase 5: Análisis Forense de Resultados Experimentales

5.1 Auditoría Perimetral de Cabeceras y Métodos con `curl.exe`

```
HTTP/1.1 200 OK
Server: nginx
Date: Tue, 10 Nov 2026 18:28:39 GMT
Content-Type: text/html
Content-Length: 1203
Last-Modified: Tue, 10 Nov 2026 04:54:42 GMT
Connection: keep-alive
ETag: "6af2a392-4b3"
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; object-src 'none'; fra
X-XSS-Protection: 0
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()

# Verificación de restricción de método HTTP (POST):
POST status: 403
```

Conclusión de Auditoría: El banner sólo reporta `nginx` (versión exacta suprimida). Las cabeceras de protección (incluida `Content-Security-Policy` y `Permissions-Policy`) están activas y las peticiones `POST` son rechazadas con `403 Forbidden`.

5.2 Prueba de Estrés DoS con Apache Benchmark (`ab -n 1000 -c 50`)

```
Document Length:        1203 bytes
Concurrency Level:      50
Time taken for tests:   4.100 seconds
Complete requests:      1000
Failed requests:        969 (Non-2xx responses: 969)
Requests per second:    243.88 [#/sec] (mean)
Time per request:       4.100 [ms] (across all concurrent requests)
```

Deducción Matemática Forense: ¿Por qué 31 OK y 969 Bloqueadas?

- **1 Petición Base:** Concedida al instante $t = 0$ según la tasa nominal (`rate=5r/s`).
- **10 Peticiones en Burst:** Absorbidas de inmediato en los 10 slots del búfer (`burst=10 nodelay`).
- **20 Peticiones por Drenaje:** Durante los **4.100 s** de la prueba (con el contenedor acotado a `0.50 CPU`), el cubo drena $\lfloor 5 \text{ r/s} \times 4.100 \text{ s} \rfloor = 20$ tokens adicionales.
- **Total Aceptadas:** 1 + 10 + 20 = **31 peticiones (200 OK)**.
- **969 Peticiones Rechazadas:** Descartadas en memoria compartida (SHM) con código **503** (`custom_503.html` de 749 bytes), verificadas en `docker logs` sin saturar el sistema.

Parámetro Comparativo	Backend / Servidor Sin Rate Limiting	Servidor Hardened (Defensa Activa Nginx)
Código HTTP ante Flood	100% del tráfico golpea al backend (riesgo de 504/502)	**503 Service Unavailable** controlado (96.9% frenado)
Exposición del Servicio	1000 peticiones llegan al aplicativo/BD	**Solo 31 peticiones (3.1%)** pasan en 4.100 s
Protección Anti-Slowloris	Vulnerable a agotamiento de sockets	`limit_conn 20` + timeouts de 10 s
Consumo de CPU / RAM	Escala con el backend y riesgo de OOM	Acotado a **128 MB RAM / 0.50 CPU** y evaluación *O*(1) en SHM

6. Diseño Estructurado de 10 Diapositivas para la Presentación

Diapositiva 1: Portada y Presentación del Proyecto

Contenido: Título del proyecto, asignatura (Sistemas de Comunicación y Seguridad Informática), nombres de los 3 integrantes, logos institucionales de Nginx, Docker y OWASP.

Diapositiva 2: Planteamiento del Problema — Escenario de Amenaza

Contenido: Vulnerabilidad de la Capa 7 frente a firewalls L3/L4. Vectores de Banner Grabbing (identificación de CVEs) y ataque de agotamiento de recursos (HTTP Flood).

Diapositiva 3: Fundamentación OWASP A05:2021 (Security Misconfiguration)

Contenido: Fuga de metadatos en cabeceras. Vectores en el cliente: Clickjacking (`X-Frame-Options: SAMEORIGIN` y CSP `frame-ancestors 'self'`), MIME-Sniffing (`X-Content-Type-Options: nosniff`), `Content-Security-Policy` y restricción de verbos (`limit_except GET`).

Diapositiva 4: Fundamentación Teórica — Defensa Activa y Tríada CIA

Contenido: Disponibilidad como pilar fundamental. Algoritmo Leaky Bucket (`5 r/s` , `burst=10` , `nodelay`), límite de conexiones concurrentes (`limit_conn 20`) y timeouts de `10s` anti-Slowloris.

Diapositiva 5: Arquitectura del Laboratorio (Stack Técnico en Docker)

Contenido: Diagrama de red Docker con la subred bridge `red-defensa (172.28.0.0/16)` . Nodo defensor Nginx Alpine (172.28.0.10, volúmenes `:ro` , límite 128 MB / 0.50 CPU) y nodo atacante Alpine con Apache Benchmark (172.28.0.20).

Diapositiva 6: Análisis de Configuración Nginx (`nginx.conf`)

Contenido: Explicación de directivas clave: `limit_req_zone $binary_remote_addr zone=zona_limite:10m rate=5r/s` (4 bytes por IP), `limit_conn` , timeouts y directiva `always` .

Diapositiva 7: Auditoría Práctica de Cabeceras y Métodos con cURL

Contenido: Ejecución de `curl.exe -I http://localhost` y prueba `POST` (→ `403`). Demostración de versión oculta y presencia de encabezados de seguridad.

Diapositiva 8: Prueba de Carga y Simulación de Ataque DoS L7

Contenido: Parámetros del ataque con `ab -n 1000 -c 50 http://172.28.0.10/`. Simulación de 50 usuarios concurrentes en ráfaga.

Diapositiva 9: Análisis Forense de Resultados (31 OK vs 969 Bloqueadas)

Contenido: Justificación matemática de las 31 peticiones `200 OK` y 969 bloqueadas con `503` (`96.9%`) en 4.100 segundos bajo evaluación $O(1)$ en memoria compartida.

Diapositiva 10: Conclusiones y Buenas Prácticas de Ingeniería

Contenido: Importancia del bastionado perimetral en Capa 7, uso eficiente de memoria compartida en microservicios e integración en pipelines CI/CD.

7. Guion de Sustentación Formal para los 3 Integrantes

> ***Expositor 1: Introducción, Escenario de Amenaza y OWASP A05:2021 (Diapositivas 1 a 3)***
> *"Estimado docente y compañeros, buenas tardes. Hoy presentamos nuestro proyecto de fin de semestre:* ***Bastionado Integral y Defensa Activa en Capa 7 sobre Nginx con Docker****.*
>
> *Al analizar la seguridad en el modelo OSI, observamos que las organizaciones blindan las Capas 3 y 4 con firewalls de paquetes, pero dejan desprotegida la* ***Capa 7 (Capa de Aplicación)****. Un atacante no necesita saturar el ancho de banda con tráfico UDP masivo; puede establecer conexiones TCP legales mediante el Three-Way Handshake y bombardear el servidor con peticiones HTTP legítimas que agotan descriptores de archivo, buffers de socket y workers.*
>
> *A su vez, mediante* ***Banner Grabbing****, herramientas automatizadas escanean el encabezado* `Server` *para correlacionar versiones exactas con vulnerabilidades CVE conocidas. En este proyecto atacamos directamente la categoría* ***OWASP A05:2021 (Security Misconfiguration)*** *suprimiendo la versión con* `server_tokens off` *, restringiendo los verbos HTTP a* `GET` */* `HEAD` *e inyectando cabeceras mandatorias con la bandera* `always` *:* `X-Frame-Options: SAMEORIGIN` *para neutralizar el* ***Clickjacking****,* `X-Content-Type-Options: nosniff` *para erradicar el* ***MIME-Sniffing****, y* `Content-Security-Policy` *junto con* `Permissions-Policy` *."*

> ***Expositor 2: Fundamentación Teórica, Rate Limiting y Arquitectura Docker (Diapositivas 4 a 6)***
> *"Para garantizar el pilar de **Disponibilidad en la Tríada CIA**, implementamos una estrategia de Defensa Activa fundamentada en el algoritmo **Leaky Bucket (Cubo Perforado)** mediante* `ngx_http_limit_req_module` *y protección anti-Slowloris con* `ngx_http_limit_conn_module`*.*
>
> *Nuestra directiva central es* `limit_req_zone $binary_remote_addr zone=zona_limite:10m rate=5r/s;`*. Destacamos el uso de* `$binary_remote_addr`*: representa la dirección IP en formato de red binario de **4 bytes** para IPv4, a diferencia de los 15 bytes que consume una cadena de texto. En un segmento de **10 MB de memoria compartida (SHM)**, Nginx tiene la capacidad de monitorear más de **160,000 direcciones IP simultáneas** en tiempo real.*
>
> *Fijamos una tasa de fuga de 5 r/s (1 solicitud cada 200 ms) combinada con* `burst=10 nodelay`*, un tope de **20 conexiones TCP simultáneas por IP** (* `limit_conn` *) y timeouts de **10 segundos** para mitigar conexiones lentas. Toda la solución corre en una red bridge aislada* `red-defensa (172.28.0.0/16)` *con volúmenes en solo lectura (* `:ro` *) y recursos acotados a 128 MB de RAM y 0.50 CPU."*

> ***Expositor 3: Demostración en Vivo, Análisis Forense de Resultados y Conclusiones (Diapositivas 7 a 10)***
> *"Procederemos ahora con la demostración en vivo y el análisis forense de la telemetría.*
>
> *En primer lugar, realizamos la auditoría de hardening perimetral ejecutando* `curl.exe -I http://localhost` *desde el host. Confirmamos que el encabezado* `Server` *reporta exclusivamente* `nginx`*, constatamos la emisión activa de* `X-Frame-Options`*,* `X-Content-Type-Options`*,* `Content-Security-Policy` *y* `Permissions-Policy`*, y validamos que una petición* `POST` *es bloqueada con* `403 Forbidden`*.*
>
> *A continuación, ingresamos al contenedor atacante y desatamos una prueba de estrés de Capa 7 utilizando Apache Benchmark:* `ab -n 1000 -c 50 http://172.28.0.10/`*, inyectando 1000 peticiones en 50 hilos simultáneos.*
>
> *(Ejecuta la prueba y muestra la consola)*
>
> *Observen las métricas forenses: de las 1000 peticiones procesadas en **4.100 segundos**, **exactamente 31 peticiones resultaron 200 OK y 969 fueron bloqueadas con código 503**. La deducción matemática coincide con el algoritmo Leaky Bucket: se concede 1 petición base en t = 0, 10 peticiones absorben los slots del* `burst=10`*, y durante los 4.100 s de la prueba el cubo drena ⌊5 × 4.100⌋ = 20 tokens adicionales (1 + 10 + 20 = 31). Las 969 solicitudes restantes fueron interceptadas y rechazadas al instante en memoria compartida.*
>
> *Esto demuestra que el servidor no colapsó ni desbordó sus límites de 128 MB de RAM y 0.50 CPU, sino que ejecutó una **Defensa Activa Exitosa en Capa 7**. Quedamos a disposición del profesor para la ronda de preguntas."*

8. Manual Rápido de Ejecución en Vivo (Cheat Sheet)

```
# 1. Ubicarse en la carpeta del proyecto y levantar el stack con Docker Compose
cd "c:\Users\Anderson Ojeda\Downloads\final2\final2"
docker compose up -d

# 2. Instalar herramientas de auditoría y ataque en el contenedor atacante
docker exec cliente-atacante apk add --no-cache apache2-utils curl

# 3. Auditoría de Cabeceras y Métodos OWASP A05 (Desde el Host en PowerShell)
curl.exe -I http://localhost
curl.exe -s -o NUL -w "POST status: %{http_code}`n" -X POST http://localhost

# 4. Ataque de Inundación DoS L7 (Desde el Atacante)
docker exec -it cliente-atacante ab -n 1000 -c 50 http://172.28.0.10/

# 5. Inspección de logs forenses en Nginx
docker logs --tail 25 servidor-defensa

# 6. Limpieza final del entorno
docker compose down
```

9. Banco de Preguntas Avanzadas para la Sustentación

Pregunta 1: ¿Por qué usar `$binary_remote_addr` en lugar de `$remote_addr` en `limit_req_zone`?

Respuesta: `$binary_remote_addr` guarda la IP en formato binario de red (4 bytes para IPv4), mientras que `$remote_addr` usa cadenas ASCII de 7 a 15 bytes. Esto optimiza el uso de la memoria compartida (SHM), permitiendo registrar 160,000 IPs en 10 MB.

Pregunta 2: ¿Por qué se procesaron 31 peticiones exitosas y no exactamente 5 con `rate=5r/s`?

Respuesta: Debido al acoplamiento de `burst=10`, `nodelay` y los 4.100 s que duró la prueba. Se aprueba 1 solicitud base + 10 solicitudes en ráfaga + $\lfloor 5 \times 4.100 \rfloor = 20$ tokens drenados durante la ejecución (1 + 10 + 20 = 31), rechazando las 969 restantes con 503.

Pregunta 3: ¿Cuál es la diferencia técnica entre devolver HTTP 503 frente a HTTP 429?

Respuesta: `503 Service Unavailable` indica indisponibilidad temporal y es el valor por defecto de Nginx para proteger el motor. `429 Too Many Requests` (RFC 6585) es el código formal para rate limiting en APIs REST.

Pregunta 4: ¿Por qué la prueba de estrés no saturó la CPU ni tumbó el servidor?

Respuesta: Gracias a la arquitectura orientada a eventos no bloqueante (`epoll`) y a la evaluación en memoria compartida *O*(1). Nginx descarta las 969 solicitudes excedentes en RAM sirviendo `custom_503.html` sin saturar un eventual backend, además de estar protegido por `limit_conn 20` , timeouts de 10 s y límites de 128 MB / 0.50 CPU.

Pregunta 5: ¿Por qué es mandatoria la directiva `always` en `add_header` ?

Respuesta: Por defecto, Nginx solo añade cabeceras en respuestas 2xx/3xx. La bandera `always` obliga a inyectar las cabeceras de seguridad también en códigos de error (4xx y 5xx), impidiendo que respuestas de error sean vulnerables a Clickjacking en un iframe.